Certified Minimum-Cost Scheduler Refinement for Stateful BFT Testing

Abstract. Sharing one admission decision across message phases can remove a BFT agreement counterexample. We formulate the least-cost phase distinctions needed to recover an unsafe execution over arbitrary control-labelled protocol transition systems. A static policy must preserve each fine key's answer across repeated queries; the least refinement of a consistent trace is exactly its required and mixed coarse fibres. Exact finite-state search returns a witness and an independently checked closed-sublevel certificate, without assuming independent pivots or additive quorum contributions. A labelled simulation transfers abstract lower bounds to a detailed model; a concrete witness of matching cost establishes its optimum. An invariant-guided abstraction procedure eliminates entire spurious policies and makes finite progress. In a seven-validator model with six interacting control fibres, 48,201 states and 55,283 transitions, nine false policies are eliminated before a 562-state quotient certifies cost 2. Every concrete edge is checked and the witness is replayed through protocol guards. Direct comparison records the full refinement overhead; smaller quotients do not imply cheaper search or proofs. Independent policy oracles and a Set Cover reduction validate the general solver, while a repair-graph specialization retains weighted quota synthesis up to 48 pivots. Lean mechanizes the policy, certificate, simulation and invariant-refinement principles. Separate tests execute unchanged CometBFT code with native signatures. The executable optimum is relative to a complete supplied model and static control interface; applying it to arbitrary client code requires transition and observation coverage.

Best student paper award: Not nominated.

Certified Minimum-Cost Scheduler Refinement for Stateful BFT Testing

Anonymous author
Anonymous affiliation

Abstract

Sharing one admission decision across message phases can remove a BFT agreement counterexample. We formulate the least-cost phase distinctions needed to recover an unsafe execution over arbitrary control-labelled protocol transition systems. A static policy must preserve each fine key's answer across repeated queries; the least refinement of a consistent trace is exactly its required and mixed coarse fibres. Exact finite-state search returns a witness and an independently checked closed-sublevel certificate, without assuming independent pivots or additive quorum contributions. A labelled simulation transfers abstract lower bounds to a detailed model; a concrete witness of matching cost establishes its optimum. An invariant-guided abstraction procedure eliminates entire spurious policies and makes finite progress. In a seven-validator model with six interacting control fibres, 48,201 states and 55,283 transitions, nine false policies are eliminated before a 562-state quotient certifies cost 2. Every concrete edge is checked and the witness is replayed through protocol guards. Direct comparison records the full refinement overhead; smaller quotients do not imply cheaper search or proofs. Independent policy oracles and a Set Cover reduction validate the general solver, while a repair-graph specialization retains weighted quota synthesis up to 48 pivots. Lean mechanizes the policy, certificate, simulation and invariant-refinement principles. Separate tests execute unchanged CometBFT code with native signatures. The executable optimum is relative to a complete supplied model and static control interface; applying it to arbitrary client code requires transition and observation coverage.

2012 ACM Subject Classification Theory of computation → Distributed algorithms; Computer systems organization → Dependable and fault-tolerant systems and networks; Software and its engineering → Formal methods

Keywords and phrases Byzantine fault tolerance, consensus testing, scheduler synthesis, certifying algorithms, phase refinement, formal methods

Digital Object Identifier 10.4230/LIPIcs.OPODIS.2026.0

1 Introduction

A fault-injection scheduler determines which behaviors a distributed-system test can observe. Twins, randomized Byzantine testing, Netrix and ByzzBench demonstrate the practical value of controlling message delivery [2, 16, 7, 1]. These controls expose a design problem: **which distinctions between admission decisions are necessary to observe an agreement violation, and what is their minimum cost?** A phase-specific policy may admit an enabling prevote and suppress a later precommit carrying repair evidence. A single admission bit for their shared peer/view key cannot make that choice. Additional evidence can repair a validator before it signs an indispensable conflicting vote.

Quorum arithmetic explains some such examples, but a general protocol has stateful dependencies. Several messages jointly enable one signature; one signature contributes to multiple local quorums; a timeout changes which proposal is legal; a commit certificate may wait for missing block data. Assigning each channel an independent pivotal contribution loses these interactions. Repeated deliveries raise another problem: a search that changes a fine key's answer at each visit implements a different scheduler from a static admission policy.

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0

We formulate refinement directly over a labelled protocol transition system. A finite set of fine decision keys is partitioned into coarse fibres. Unsplit fibres share one Boolean admission value; a selected fibre may distinguish its fine keys. Every fine key retains its value throughout the execution. Protocol state, message generation, locking, quorum assembly and event order otherwise remain in the transition relation. Unequal nonnegative integer costs and allowed/required refinements are explicit inputs.

Two different refinements meet in this problem. **Scheduler-interface refinement** exposes additional independent decisions and enlarges the tester's policy family. **State-abstraction refinement** separates abstract states and removes executions that the concrete protocol cannot realize. We connect these operations while keeping the protocol, unsafe predicate and environmental bound fixed. A cheap abstract counterexample is useful only as a lower bound until a compatible concrete execution attains its price.

The useful engineering output is a control set, a complete static policy and a replayable regression scenario, together with evidence that every cheaper permitted control set is insufficient in the represented domain. This can prioritize which message distinctions to expose, compare restricted harness configurations and distinguish certified bounded infeasibility from unsuccessful search. Supplied control costs express preferences or budgets; they are not measured development effort or execution time.

1.1 Contributions and assurance boundary

First, we characterize the exact least refinement of any consistent finite execution: it consists of required fibres and the fibres in which that execution observes both admission values. This result has no independence or quorum-additivity hypothesis. A product of protocol state and persistent partial policy yields exact minimum-cost unsafe reachability for every supplied finite transition system, including cyclic systems. Its portable certificate combines an unsafe execution with a closed set excluding every cheaper unsafe execution. A separate checker regenerates all relevant successors from the bound input; a search limit produces unknown.

Second, we connect model optimization to a more detailed implementation model. A labelled forward simulation transfers an abstract lower bound to every covered concrete execution. A concrete unsafe witness attaining that bound proves concrete optimality, without requiring all abstract witnesses to lift. We implement an iterative procedure: optimize an existential quotient, test its entire fixed policy on the concrete system, and refine the quotient using the reachable-state invariant of a false policy. This eliminates that policy globally. Each failed iteration strictly refines a finite partition.

Third, guarded four- and seven-validator models make controls interact through shared local quorums, persistent messages and subsequent round changes. The larger instance has six controlled endpoint pairs, two phases per pair and two replicas whose votes must jointly support both observer certificates. Complete enumeration covers each declared bounded opportunity language. Independent total-policy and permission oracles check the optima; direct product search and certified abstraction refinement use identical inputs with explicit accounting for all iterations and checking. The previous repair-graph algorithm remains an efficient structural specialization, with weighted quota certificates and cases up to 48 pivots.

Lean mechanizes policy completion, persistent-memory updates, closed-set certificates, local simulation transfer and invariant-guided quotient refinement, alongside the existing graph/order and Bellman proofs. The semantic theorems quantify over arbitrary transition systems; finite explicit models make the algorithms executable. Application to arbitrary client code still requires a faithful control-labelled model or a proved covering simulation. Our checked transition bridge is between explicit models. Separate ordinary-operation tests

of unchanged CometBFT code support selected protocol obligations.

2 Protocol semantics and the control interface

Let $P = (S, s_0, \to, B)$ be a labelled transition system with unsafe states $B \subseteq S$. In an agreement test, B means conflicting decisions by the designated correct observers. State includes whatever the protocol needs: signed messages and their availability, vote sets, locks, valid blocks, timers, membership and decisions. These fields need not decompose by validator. A delivery must consume an existing envelope; those legality conditions belong to $\to$.

Let F be a finite set of fine admission keys, K a finite set of coarse fibres, and $\kappa : F \to K$. A transition is either internal, $s \xrightarrow{\epsilon} t$, or records an actual policy query, $s \xrightarrow{(f,b)} t$, with $b \in \{0, 1\}$. For phase refinement, f can contain height/view, phase and transport endpoints, while κ forgets phase. A relayed vote's transport sender may differ from its signer; certificate support must count signers. Other deliveries and local actions may be internal relative to this selected interface.

The **control-completeness obligation** is that all dependence on the selected policy appears through these queries. A fixed policy cannot also change an unlabelled guard behind the model's back. An atomic step reading several keys can be expanded into successive queries through private intermediate states that preserve its atomicity. If an envelope was never emitted, an opportunity for it may stutter without querying admission. The completed policy still assigns that unobserved key.

A total static policy is $a : F \to \{0, 1\}$. A path is compatible with a when every visible label (f, b) satisfies $a(f) = b$. Internal nondeterminism and state-dependent enabled actions remain unrestricted within P. Thus the objective is existential: a policy and an execution reaching B. It is not a strategy winning against every environment choice, nor a liveness/fairness property.

For split set $T \subseteq K$, require $a(f) = a(g)$ whenever $\kappa(f) = \kappa(g) \notin T$. Let $Q \subseteq D \subseteq K$ be required and allowed splits, with $Q \subseteq T \subseteq D$. Assign costs $c_k \in \mathbb{N}$, including zero, and $c(T) = \sum_{k \in T} c_k$. Required keys are paid even if an execution never queries them. Costs describe supplied adapter/control choices; they are not measured network or CPU costs.

Define $\mathrm{Opt}(P)$ as the minimum $c(T)$ among admissible policies and finite compatible unsafe paths; set it to ∞ if none exists. The general semantics permit infinitely many protocol states. Our executable input represents a complete finite graph and a finite interface. A bounded exporter specifies the environment, heights, rounds and opportunities it represents; its bound is part of the theorem instance.

3 Exact refinement without independent pivots

3.1 Least refinement of a trace

For a finite path τ, let $\mathcal{O}_\tau \subseteq F \times \{0, 1\}$ be its observed queries. Call τ consistent if no fine key occurs with both values. Define its mixed fibres by

$$M(\tau) = \{k : \exists f, g,\ \kappa(f) = \kappa(g) = k, (f, 0), (g, 1) \in \mathcal{O}_\tau\}.$$

Consistency remains necessary when every fibre is split: splitting distinguishes phases, not successive uses of the same fine key.

Theorem 1 (exact trace refinement). A path τ is realizable under split set T exactly when it is consistent and $M(\tau) \subseteq T$. Under allowed/required constraints, a refinement exists

exactly when τ is consistent and $Q \cup M(\tau) \subseteq D$. In that case $T_\tau = Q \cup M(\tau)$ is least by inclusion and minimum in nonnegative sum cost.

Proof. A realizing static policy gives one value to every fine key. If an observed fibre has both values, uniformity forces it to be split. Conversely, in each unsplit fibre use its single observed value, or 0 if none was observed. In a split fibre use each fine key's observed value, or 0 if unseen. Consistency makes these choices well-defined and they complete a total realizing policy. Every feasible split set contains $Q \cup M(\tau)$, which is itself feasible when allowed. Nonnegative costs preserve inclusion. □

This characterizes the least refinement of a path, rather than prescribing which path to find. The path can contain mutually dependent quorums, alternative support, recovery, repeated messages and cycles. Such behavior changes which paths are legal; it does not change the policy-completion argument.

3.2 Protocol state times persistent policy memory

Use product states (s, μ), where $\mu : F \to \{\bot, 0, 1\}$ records unseen or previously selected answers. Initially μ is everywhere $\bot$. An internal edge keeps μ. An edge (f, b) is enabled only if $\mu(f) \in \{\bot, b\}$; it writes $\mu(f) = b$. Reject a successor if its mixed fibres, together with Q, are not contained in D. Define

$$g(s, \mu) = c\big(Q \cup M(\mu)\big).$$

Memory only accumulates observations, so g never decreases. The initial cost is $c(Q)$, and each newly mixed fibre contributes its cost once. Repeating the same admission or revisiting protocol state does not repay a split. Crucially, merging (s, μ_1) and (s, μ_2) merely because they share s can lose future consistency constraints.

Theorem 2 (general minimum-cost synthesis). For a finite explicit P, the minimum g of a reachable unsafe product state is exactly $\mathrm{Opt}(P)$. If there is no such state, no admissible policy exposes unsafety in P.

Proof. Induction on an execution shows that its visible queries lift to the product exactly when they are consistent and their required/mixed union is allowed. The final memory records precisely those queries. Theorem 1 constructs a total policy at cost g for every unsafe product path. Conversely, every unsafe execution under an admissible split set lifts and has $g \le c(T)$. Taking minima gives both inequalities and the infeasibility equivalence. □

A priority queue explores product states in increasing g. Because g is determined by the state, revisiting it cannot yield a cheaper price; the first popped unsafe state attains the minimum. There are at most $|S|3^{|F|}$ states. The implementation indexes outgoing input edges and supports internal cycles, repeated labels, zero costs and nondeterministic successors. Exhausting a configured state or time budget returns unknown, never infeasible.

Worst-case exponential behavior is expected. The cost-bounded decision problem is NP-complete already on acyclic input graphs with two fine keys per fibre. A weighted Set Cover instance is represented by first querying one phase negatively for each set, then selecting a positive second-phase edge for a covering set at every element layer. The path pays each used set once. Appendix A gives the reduction and polynomial verification argument. This relates the problem to established minimum-label and specification-revision optimization [8, 9, 10]; the product search is a generic exact procedure, not a polynomial-time claim.

3.3 Independent lower-bound certificates

An unsafe witness proves only an upper bound. For a proposed optimum C, the certificate also supplies a set Z of product states strictly below C. The checker validates:

1. Every state in Z has cost below C and is not unsafe.
2. The initial product state belongs to Z whenever its cost is below C.
3. For every state in Z, **every input edge** inducing a legal successor of cost below C leads to another state in Z.

The checker reconstructs these successors with separate policy-update code. It does not trust an optimizer-supplied edge list or invoke the optimizer. A SHA-256 binding identifies the embedded complete input; an externally supplied expected input prevents substitution of another model. The witness is separately replayed, including total-policy uniformity and its actual paid cost.

Theorem 3 (closed-set certificate soundness). The three obligations imply that every admissible unsafe execution costs at least C. A valid unsafe witness of cost C is optimal. A bad-free set containing the initial state and closed under all legal product successors, without a cost cutoff, certifies infeasibility.

Proof. Suppose an unsafe product path ends below C. Nondecreasing cost places its entire prefix below C. The initial and closure obligations inductively put every prefix state in Z, contradicting terminal unsafety. Theorem 2 transfers this lower bound to admissible policies; the witness attains it. Without a cutoff, the same induction excludes every unsafe path. □

The exact reachable strict sublevel is always a suitable certificate when C is optimal. Certificates can nevertheless be exponentially large. Checker limits also return unknown. An omitted reachable successor below the claimed bound, an inconsistent repeated answer, an altered expected input or an expensive substituted witness is rejected. Closed-set reasoning and independent witnesses follow the certifying-algorithm methodology [13]; the contribution is their connection to static scheduler refinement and protocol-labelled reachability.

4 Transferring optimality across model boundaries

4.1 Abstract lower bound and concrete witness

Let P_C be a concrete transition model and P_A an abstraction with the same fine keys, fibre map, allowed/required sets and costs. A state map $h : S_C \to S_A$ covers the initial states and satisfies two local obligations. Each concrete controlled transition has an abstract transition with exactly the same (f, b) label. Each concrete internal transition has an abstract internal match or maps to a stutter. Moreover $s \in B_C$ implies $h(s) \in B_A$. More general finite internal matches are possible; the executable checker verifies this one-edge/stutter sufficient case on every concrete input edge.

These conditions preserve each finite execution's complete visible query sequence. No assumption that an abstract unsafe path has a concrete realization is needed for the lower-bound direction. In particular, merging states may create spurious abstract paths.

Theorem 4 (implementation-model optimality transfer). Under the local coverage conditions,

$$\mathrm{Opt}(P_A) \;\le\; \mathrm{Opt}(P_C).$$

Consequently a certified abstract lower bound L and a concrete admissible unsafe witness of cost U give

$$\boxed{L \leq \mathrm{Opt}(P_C) \leq U.}$$

If $L = U$, the witness is optimal in the concrete model. Abstract infeasibility implies concrete infeasibility.

Proof. Simulate an arbitrary concrete path step by step, omitting only internal stutters. The abstract path has the same visible queries and hence the same realizing policies and required/mixed cost. Concrete unsafety maps to abstract unsafety. Thus every concrete witness gives an abstract witness costing no more, and every certified abstract lower bound applies to the concrete witness. A real concrete witness supplies the upper bound. The infeasibility statement follows by contradiction. □

The lower-bound direction needs coverage of **all** relevant concrete executions. Matching one previously found trace establishes no such coverage. Conversely, a forward overapproximation alone supplies no concrete upper bound. This distinction allows a useful middle ground: an abstraction may contain many spurious behaviors and still suffice to prove optimality when one concrete witness matches its certified lower bound.

The checker validates every supplied concrete edge, the initial-state map, unsafe-state inclusion and equality of the scheduler interface. Erasing a controlled admission as if it were an internal stutter is forbidden. Our executable bridge therefore establishes a genuine transition correspondence between finite models. The Lean local-simulation theorem itself has no finite-state restriction; using it for an infinite implementation requires a proved relation covering that implementation, rather than a finite test sample.

4.2 Refining a false policy

We use existential quotients to search for a sufficient abstraction. For a partition h, quotient edges are projections of concrete edges and a quotient state is unsafe if it contains any concrete unsafe state. Start with any partition, including one group. Solve the quotient and check its certificate. If it is infeasible, Theorem 4 finishes. Otherwise let a be the returned total policy. Compute ordinary concrete reachability using only edges compatible with a.

If a concrete unsafe path exists, its admissible cost is at most the quotient's optimum, because a is realizable using the returned split set. Theorem 4 gives the reverse inequality; replay therefore certifies the concrete optimum. If no such path exists, its full concrete reachable set R_a contains the initial state, is closed under a-compatible edges and excludes unsafety. Refine every block by

$$h'(s) = \big(h(s), \mathbf{1}_{s \in R_a}\big).$$

Theorem 5 (policy elimination and finite progress). The refined existential quotient has no unsafe path compatible with the rejected policy a. If the old quotient had such a path, at least one old block splits. On a finite concrete state set, starting from partition h_0, the procedure uses at most $|S_C| - |h_0(S_C)|$ failed refinement iterations before returning an optimal witness or infeasibility, unless an explicit resource limit intervenes.

Proof. A refined block lies wholly inside or outside R_a. Any compatible quotient edge leaving a block inside R_a has a concrete representative whose source lies in R_a; closure puts its target there too. The initial quotient state is inside, and no inside block is unsafe. Thus

every compatible quotient path stays safe. If the old partition already had this homogeneity, the same argument would exclude its supposed unsafe path; therefore some block must split. Each failure increases the number of nonempty blocks, which is bounded by $|S_C|$. At a discrete partition the quotient is isomorphic to the concrete graph, so no false policy remains. This proves the bound and eventual completion. □

This is a specialization of counterexample-guided abstraction refinement [3]. The refinement invariant rejects an **entire static policy**, rather than only the returned path. The next abstract optimum cannot decrease because partition refinement removes abstract behaviors while preserving all concrete ones. The certificate history checks each intermediate bound, every safe policy invariant and every strict partition refinement. The final all-edge bridge and matching witness suffice for the optimality claim independently of that history.

The prototype takes a complete finite concrete graph as input. Its quotient can greatly reduce the product search, but it does not avoid constructing that graph. Symbolic invariants or code-level simulations could replace explicit enumeration in the theorem; they are not supplied automatically by this implementation.

5 Independent pivots as a structural specialization

The general result subsumes arbitrary state dependencies, while a stronger behavioral contract permits a more structured algorithm. Let pivots $I = \{1, \dots, k\}$ have indivisible positive weights w_i, with $w(X) = \sum_{i \in X} w_i$. Two certificate families need deficits d_0, d_1 beyond fixed support. Pivot i emits its first-family signature if and only if its enabling message E_i is admitted; no other action substitutes for that contribution. Once the first certificate is feasible, that pivot supplies its second signature at V_i exactly when admitted repair R_i was not processed earlier. Initially ready closed pivots can still sign second. No other transition changes this test; repair cannot withdraw signed output. Each pivot has its own coarse key coupling E_i, R_i: unsplit admission accepts both or neither, while splitting permits distinct choices. In every first-certificate-feasible execution, an admitted repair is available and processed at its named slot. Both certificate availability and designated unsafe observations must satisfy this coverage contract.

A finite opportunity partial order includes **all** linear extensions, retains suppressed slots and has distinct event kinds $R_i \neq V_j$ for every i, j. Use transitive precedence to define $i \to j$ in repair graph H exactly when $R_i \prec V_j$. Jointly delaying repairs for J is possible exactly when $H[J]$ is acyclic. The alternating-cycle/order-reversal proof and the finite schedule construction are mechanized; full statements and proofs appear in Appendix E [15].

Theorem 6 (weighted refinement characterization). Under this independent-pivot contract, an admissible refinement is feasible exactly when there is a partition

$$I = T \,\dot\cup\, J \,\dot\cup\, X \,\dot\cup\, Y, \quad Q \subseteq T \subseteq D, \quad H[J] \text{ acyclic},$$

$$w(T \cup J \cup X) \geq d_0, \qquad w(T \cup J \cup Y) \geq d_1.$$

Here T is split, J unsplit and reusable, X open with first support, and Y closed with second support. Minimize $c(T)$ over these choices.

Proof. A split key can be normalized to admit E_i and suppress R_i, supplying both signatures under the contract. In an unsafe execution, partition the remaining keys into open reusable pivots J, other open pivots X, and closed pivots Y. The actual order implies

acyclicity of $H[J]$ and the support inequalities. Conversely, choose a legal order reversing each (R_i, V_i) for $i \in J$, open $T \cup J \cup X$, and suppress repairs in T. The guaranteed supports satisfy both thresholds; additional second signatures from X only help. □

For unit weights, writing $\alpha(H) = \max\{|J| : H[J] \text{ acyclic}\}$, coarse unsafety is equivalent to $\max(d_0, d_1) \le k$ and $d_0 + d_1 \le k + \alpha(H)$. If $Q = \varnothing$, $D = I$ and every split has unit cost, the exact minimum is

$$t_* = \max(0, d_0 + d_1 - k - \alpha(H)), \qquad \max(d_0, d_1) \le k.$$

Unequal weights require actual subset support; aggregate capacity is insufficient. The structural solver enumerates four-role assignments within strongly connected components and combines their two supports with quota dynamic programming. Separate Bellman inequalities certify every local alternative, and replay supplies the matching witness. Enumeration is exponential in the largest remaining component and pseudo-polynomial in integer quotas. Saved cases reach 48 pivots; configured limits return unknown. An explicit adapter also enumerates the same enabling, repair and immutable-signature semantics as a general LTS. Both solvers agree with an independent raw admission/order oracle on 1,507 cost/quota/permission instances, including all 219 labelled four-event partial orders. Every feasible general witness normalizes to the structural admission pattern at equal cost, providing an executable check of this specialization.

The reusable capacity $r(T) = |T| + \alpha(H[I \setminus T])$ is not submodular. With edges $0 \leftrightarrow 1$ and $0 \leftrightarrow 2$, full quotas $(3, 3)$ and costs $(M, 1, 1)$ for $M > 2$, both leaf splits jointly suffice for cost 2 although neither has an immediate gain. The only positively improving singleton is the centre, costing M. Immediate-positive-gain-per-cost selection thus has ratio $M/2$, unbounded in this family. This concerns that specified myopic rule. Ten disjoint copies give certified optimum 20 versus its cost 1000 in the saved study.

6 Stateful protocol instance and experiments

6.1 A coupled guarded model

The executable rule model follows the pinned CometBFT v0.39.3 source `49b82838fcca44 2b2445f76605c101609ed04130` [4, 5]. At one height, take $n = 4$, quorum 3, Byzantine identity B and replicas A, P, C. The explicit mutant omits only P's round-0 persistent lock assignment; it retains valid-block state and normal later lock updates. The baseline omits nothing. Unsafety requires **the two unmutated observers** A, C to decide different blocks. A disagreement involving only the defective replica is insufficient.

The two controlled round-0 fibres are $B \to P$ and $A \to P$, costing 5 and 2. Fine keys include all relevant proposal, prevote, precommit and block phases on these fibres. In particular the relayed $A \to P$ proposal is controlled, not silently exempted from coarse uniformity. All other transport admissions follow the declared fixed policy. Model messages use deterministic HMAC test authentication. The Byzantine process supplies explicitly signed test messages; every correct vote is emitted by a local guard and quorum certificates count distinct actual signers.

The A, B prevotes jointly complete P's local quorum with its own vote. Their precommits jointly complete its repair certificate. Both channels therefore act on the **same** quorum and persistent state; they cannot be represented as independent contributors, one per control fibre. Later message generation depends on proposal availability, mixed-vote timeout guards, locks and valid blocks. The complete environment has two rounds, 12 stages and 32 once-only

opportunity slots, including an unordered repair/precommit-timeout group and a guarded missing-block delivery. It permits every permutation within each declared stage and includes conditional no-op opportunities for envelopes or timers not yet produced or enabled at that opportunity. It is enumerated completely; a state limit aborts export rather than returning a supposedly safe frontier.

In the minimum witness, separating the cheaper $A \to P$ fibre preserves the enabling proposal/prevote while withholding the decisive repair contribution. A obtains the first actual certificate and decides 0. A genuine mixed precommit quorum permits round advancement without committing P to 0. With both peer precommits suppressed, the first certificate can still form at A, but P lacks even a mixed quorum and cannot advance: absence of repair does not imply second-vote readiness. One peer precommit can enable progress when the other is withheld, yet complete repair when both are present. Unlocked proposer C proposes 1; the unlocked mutant P can support it even while retaining valid block 0. The locally collected second quorum makes C decide 1. Appendix C details the original guarded trace; the coupled artifact records every action of the optimized witness and replays each successor through the protocol code.

Repair is tested against **signature issuance**. Once the indispensable conflicting precommit has been emitted, a later repair cannot remove it from another observer's future certificate. Retained controls explicitly produce $A = 0, P = 0, C = 1$ after such a late repair. A separate relay schedule preserves FIFO-compatible admitted vote-envelope order. Those timing controls use stated gossip languages and are not executions of the production P2P reactor.

6.2 General synthesis and checked abstraction

The general study compares complete graph reachability under every total policy and every allowed split subset with the product optimizer. It covers 7,904 graph/cost/permission configurations and six semantic examples: repeated-key inconsistency with a cycle, a three-phase split paid once, state-dependent cost alternatives, required unused fibres, zero-cost splitting and a spurious state merge. These challenge assumptions that the independent-pivot solver does not address. Five protocol controls distinguish quorum-enabled timeout ordering, block admission and failure of initial readiness: the same fixed fine policy has both safe and unsafe opportunity orders, and commit waiting ends only after an emitted block is received.

A second standalone oracle compares both direct synthesis and iterative abstraction on 2,960 instances: 400 seeded finite systems, 2,048 exhaustive edge/cost/constraint configurations and all 512 ordered three-set cover families on three elements. It checks 5,920 optimum comparisons and 6,857 certificates across all abstraction iterations, with zero mismatches. Its ordinary-reachability oracle does not use product-state updates or a mixed-fibre cost formula; it enumerates total policies and split subsets explicitly. Tests additionally reject deleted input edges, missing closed-set successors, invalid repeated answers and substituted higher-cost witnesses, and disable optimizer helpers while checking certificates.

For the coupled protocol, the exporter is independently challenged by 128 direct protocol explorations: all 64 total policies in both baseline and mutant programs. These regenerate successors from guarded protocol code and compare complete reachable state and edge counts with the exported quotient after forgetting the same first-arrival diagnostic field. Replaying the synthesized path checks every corresponding semantic state, authentication, actual quorum evidence and the two observer decisions. This tests the exporter separately from optimizing its graph.

The final quantitative state counts and abstraction history are supplied in Table 1.

Model / initial abstraction	States / edges	Final groups	Iterations	Optimum
Coupled mutant / one group	2,516 / 5,856	26	12	2
Coupled mutant / audit field	2,516 / 5,856	1,978	1	2
Baseline / one group	2,372 / 5,574	1	1	∞

Table 1 Checked finite-model optimality. The coarsest-start mutant run eliminates 11 false policies; every concrete edge is covered. Baseline has no unsafe state in its complete bounded graph. The audit-field quotient is a separate projection check.

The coarse-start procedure repeatedly rejects unrealizable policies before its lower bound is attained by a concrete witness. The audit-only quotient, which forgets a first-arrival diagnostic field, is a separate check of projection correctness. Its compression ratio is not a claim about minimizing a production client's natural state representation.

The coarsest-start mutant quotient grows through $1, 2, 4, 6, 8, 11, 14, 16, 19, 21, 24, 26$ states. Its lower bound is 0 for four iterations and 2 thereafter; only iteration 12 supplies a concrete matching witness. The final quotient has 60 edges. The actual witness processes 30 opportunities, records 172 protocol events and 16 signed model messages, and passes 12 guard/evidence audits. With only the expensive fibre refinable the optimum is 5; requiring both costs 7; forbidding all splits is infeasible. These prices are declared inputs.

6.3 A larger experiment with six interacting controls

The second guarded instance has seven validators, quorum five, Byzantine identities B_0, B_1, pivots P_0, P_1 and unmutated observers A, C (with D supplying additional correct votes). Only the two pivots' round-0 lock assignments are omitted in the mutant. Six fibres control round-0 vote deliveries from A, B_0, B_1 to each pivot; PREVOTE and PRECOMMIT are their twelve fine keys. Proposals and other transport endpoints follow an explicitly fixed policy. The experiment optimizes this vote interface, not every message class on every endpoint.

These controls affect shared protocol state. A pivot needs all three controlled prevotes together with its own and the other pivot's vote to form quorum five. In the later phase, received precommits must supply enough support to enable a timeout without completing the old matching commit certificate. Suppressing every repair contribution can prevent round advancement. Both pivots' actual signatures contribute to the same first and second observer certificates. Thus the six controls do not satisfy the independent-contribution and initial-readiness contract of Theorem 6.

The environment contains one height, two rounds, 16 stages and 23 once-only opportunities. Fixed macro sequences expand into primitive guarded events; all permutations inside the declared variable groups are explored. Delivery/timeout races are real: one fixed policy has both safe and unsafe orders. Late matching repair after round advancement can enter commit waiting for an absent old block and prevent further signing. These bounds exclude arbitrary cross-stage interleavings. No diagnostic history field inflates the state space.

The complete mutant has **48,201 states, 55,283 transitions and 36 unsafe states**. Unsafety still requires A, C to disagree. Baseline and single-pivot-mutation graphs have no unsafe state. Removing cross-pivot initial support, nil-vote timeout support or a necessary second-round pivot vote also excludes the disagreement. The minimum witness is replayed through the primitive guards, including authentication, unique signatures, actual signer sets, locking, timer enabledness and irreversible decisions; it records 26 signed messages, ten local certificates and 431 primitive events, with 19 audits passing. Appendix G gives the complete

Unit-cost mutant: measured work	Direct	Certified refinement
Final graph states	48,201	562
Discovered product states, all iterations	21,544	121,615
Peak product states in one solver call	21,544	21,627
Final lower-bound proof states	7,103	7,130
Charged time (s), median of three repeats	6.194	13.528
Exact minimum cost	2	2

Table 2 The larger coupled instance validates abstract-to-concrete optimality, but does not show a search or proof-size advantage. Refinement work includes every intermediate product search; concrete generation, policy exploration and checking are charged separately in the artifact. A quotient-state reduction is not a total memory reduction.

interface and accounting boundary.

We compare exact direct search with certified refinement on identical graphs, prices and permissions. A guard-aware initial quotient remembers control-flow progress, unsafe observations, each pivot's local matching-prevote count and its emitted round-0 precommit. It forgets incoming precommits, timers, locks and commit state. These features depend on protocol state, not on the desired policy or optimum. This seed was introduced after a slow flow-only development pilot; the weaker start is retained as a separately bounded sensitivity experiment. No general automatic abstraction-discovery claim is made.

For unit costs, the initial guard-aware quotient incorrectly suggests a zero-cost concrete violation. Its returned all-admit policy has no unsafe concrete execution. Whole-policy refinement removes nine such false candidates in ten iterations; abstract lower bounds are $0, 1, 1, 1, 1, 1, 1, 2, 2, 2$. The final 562-state quotient supplies a certified bound of 2 and a concrete witness of cost 2. Merely accepting the initial abstract witness would give the wrong answer; the all-edge bridge, safe-policy invariants and matching replay explain the role of certification and refinement.

Table 2 gives a deliberately full accounting. Refinement performs more cumulative product work and its final lower-bound set is slightly larger. The final bridge additionally contains the complete concrete-to-abstract map, while its history retains all rejected-policy invariants. For this materialized instance direct search is the simpler baseline; the experiment supports a sound route from an initially misleading abstraction to an exact concrete-model result. Each timed total sums separately measured graph construction, partition construction, solution generation, independent checking and guarded replay; oracle enumeration, serialization and disk writes are excluded symmetrically. These are component sums, not contiguous workflow timings. The flow-only seed remains unknown after eight refinements, as do both methods under a 10,000-product-state per-call cap.

An independent total-policy BFS covers all 4,096 policies for each of three guard variants and evaluates 384 reused cost/permission configurations. Unit costs yield nine minimum pairs, one incoming fibre per pivot; costs $(1, 2, 3, 1, 2, 3)$ select the two A fibres uniquely. Forbidding every P_1 split is certified infeasible. These are concrete interface-design answers under supplied prices, not measured implementation savings.

6.4 Structural and native controls

The repair-schema studies remain complementary evidence. A raw-event oracle examines all 530 directed graphs with loops for $k \leq 3$, 50,363 legal orders and 1,357,209 executions. It checks 103,460 fixed-split boundaries and 35,223 weighted optimum/infeasibility certificates.

A second oracle over 48 arbitrary six-event DAGs examines 4,647 orders and 72,117 executions, checking another 3,253 optima. There are no mismatches. These are finite encoding checks, not counts of distinct protocols. Thirteen of 15 structural scale cases have certified optima, including 48-pivot instances; two exceed the declared component limit 10.

Separately, four ordinary-operation scenarios execute the unchanged native consensus package of the same pinned CometBFT revision with Go 1.25.0. They cover normal commit/missing-block catch-up, immutable signed output, quorum-enabled timeouts, persistent locking and valid-block proposal reuse. Four native validators produce 32 Ed25519 votes and 132 recorded events, passing 14 checks. A separate Python/OpenSSL parser verifies every signature and its canonical metadata; three normal repeats and one race-detector run also pass. Source files are checked unchanged before and after the test overlay.

These native scenarios are not a native reproduction of the mutant disagreement or a proof of the new model-to-client simulation. They dispatch internal events synchronously using upstream test keys and in-memory fixtures. Reactor, network, WAL and persistent signer behavior are outside their scope. Native timestamps remain untouched, so semantic summaries are compared while raw signatures are independently revalidated.

7 Formal assurance, related work and reproducibility

7.1 Mechanized and executable obligations

The Lean 4.33.0 project checks 46 selected declarations. The new module constructs total realizing policies, proves least required/mixed splits, and equates persistent-memory query guards with consistency of extended observations. It proves monotone trace cost, strict-sublevel lower bounds, matching-witness optimality, full-closure infeasibility, and visible-trace coverage from local labelled simulation. Its invariant-splitting theorem excludes all bad quotient paths of a rejected policy and proves that a spurious old policy forces a genuine split. The finite iteration-count bound is a paper-level cardinality argument.

These semantic proofs require neither finite protocol state nor independent pivots. Independent kernel rechecking and an axiom audit accompany the build. Labelled trace coverage, full-closure infeasibility and invariant-preserving quotient soundness use no axioms; other declarations use only the reported standard Lean axioms. The earlier graph/order realization and Bellman modules remain integrated. The completeness of the Python checker, the whole optimizer implementation and the protocol exporter are not mechanically verified. Their distinct executable oracles and negative controls support those remaining obligations.

Earlier Boolean Lean, SMT, Python and Go checks remain available. Agreement of 384 Python/Go rows for $f = 1, \ldots, 64$ checks an earlier terminal classifier. The historical declaration named `bounded_tendermint_refines_quorum_pivot` establishes that classification, not a transition refinement. The new all-edge bridge is separately identified in the claim ledger.

7.2 Relationship to prior methods

Communication-closure testing compares asynchronous and lossy synchronous executions under explicit implementation conditions [6]. Its communication rounds distinguish protocol steps. Coupling admission across those steps is an additional operator; our negative examples do not refute its normalization theorem. State observations also require careful treatment in action-based reductions, as the inconsistent-labelling literature demonstrates [14].

Twins already discusses certificate arrivals advancing stale twins and supplies message-type filtering in its executor [2]. ByzzFuzz, Netrix and ByzzBench supply randomized testing, programmable execution and benchmark infrastructure; Netrix also constructs tests from protocol clauses and invariants [16, 7, 1]. FEST adapts schedule generation using behavior and scenario feedback, including scenario constraints and compliance feedback [11]. These methods establish that semantic controls and guided exploration are practical. Our optimization asks which least-cost permitted distinctions the interface must expose, and certifies insufficiency of every cheaper choice for the fixed safety objective. No comparative bug-finding-speed claim follows.

Product-state reachability, simulation with internal actions, independently checked witnesses and abstraction refinement are established techniques [12, 13, 3]. Minimum-label paths charge distinct encountered labels, while minimal specification revision changes a specification to recover realizability [10, 9]. Here the protocol and unsafe property remain fixed: the optimized object is the partition of static admission decisions. Persistent fine-key consistency constrains legal paths, and a coarse fibre is paid when observed answers differ or its split is required. The specific contribution connects that semantics to complete lower-bound certificates, all-edge cost transfer and whole-policy invariant elimination. The structural specialization further connects order reversal to weighted quorum deficits. These are scheduler-specific results built on the cited methods; general CEGAR, shortest-path search and simulation are not new claims.

7.3 Scope and reproduction

Optimality now covers every execution of a supplied complete finite transition model under its declared static control interface. Interdependent locks, alternative quorum support and state-dependent message generation are permitted. Recoveries, changing membership or additional phases can be included as state and transitions; the artifact does not instantiate every such extension. Changing the interface to adaptive decisions, dropping legal transitions, omitting controlled queries or treating an unexpanded frontier as terminal changes the problem and invalidates the corresponding claim.

For arbitrary unbounded client programs, a terminating complete reachability optimizer cannot be expected in general; Appendix A gives the direct halting reduction. Nevertheless the semantic transfer theorem can cover an infinite concrete implementation when its local simulation and cost/observation obligations are proved. The present source-backed guarded model and native conformance tests do not discharge that universal client correspondence.

The identifying artifact URL is omitted from this anonymous review copy. Running `./reproduce_revision.sh` executes all general and structural studies, both forms of independent policy oracle, guarded exporter/replay checks, native conformance/signature/repeatability checks, Lean build and axiom audit, and the LIPIcs build. Mandatory failures stop the run. Input hashes, complete certificates, raw observations, claim mappings and tool-version logs accompany the paper. Deterministic results are compared to committed references; resource limits remain explicit unknown results. An empty dependency cache requires network downloads.

8 Conclusion

Minimum-cost phase control can be proved directly from protocol transitions and persistent admission decisions. The exact mixed-fibre characterization supports arbitrary state dependencies, while closed sublevel certificates establish that no cheaper policy exposes unsafety.

Local simulation transfers that lower bound to a more detailed model, and a matching concrete witness closes the optimality argument. Policy-invariant refinement provides a constructive way to remove misleading abstract behaviors. Independent-pivot graphs remain a useful structural acceleration within this broader framework. The remaining implementation obligation is explicit transition, control and observation coverage for the client and execution language being claimed.

8.1 Acknowledgments and AI-use disclosure

An OpenAI language model assisted with research organization, literature discovery, proof exploration, manuscript drafting, code generation and consistency checking. The author retains responsibility for correctness, originality, citations and experimental claims. The model is not an author.

References

1. Jo ao Miguel Louro Neto and Burcu Kulahcioglu Ozkan. A benchmark framework for byzantine fault tolerance testing algorithms. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *Open Access Series in Informatics (OASIcs)*, pages 13:1–13:11, 2025. doi:10.4230/OASIcs.FMBC.2025.13.
2. Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. Twins: BFT systems made robust. In *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, volume 217 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:29, 2022. doi:10.4230/LIPIcs.OPODIS.2021.7.
3. Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement for symbolic model checking. *Journal of the ACM*, 50(5):752–794, 2003. doi:10.1145/876638.876643.
4. CometBFT Project. Cometbft consensus specification. Release v0.39.3, commit prefix 49b8283, 2026. Engineering specification pinned for the bounded instantiation. URL: https://github.com/cometbft/cometbft/blob/v0.39.3/spec/consensus/consensus.md.
5. CometBFT Project. CometBFT consensus state implementation, 2026. Release v0.39.3. Exact source snapshot and integrity manifest included in the artifact. URL: https://github.com/cometbft/cometbft/blob/49b82838fcca442b2445f76605c101609ed04130/consensus/state.go.
6. Cezara Drăgoi, Constantin Enea, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Filip Nikšić. Testing consensus implementations using communication closure. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):210:1–210:29, 2020. doi:10.1145/3428278.
7. Cezara Drăgoi, Srinidhi Nagendra, and Mandayam Srivas. A domain specific language for testing distributed protocol implementations. In *Networked Systems (NETYS 2024)*, Lecture Notes in Computer Science, pages 100–117, 2024. doi:10.1007/978-3-031-67321-4_6.
8. Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi:10.1007/978-1-4684-2001-2_9.
9. Kangjin Kim, Georgios E. Fainekos, and Sriram Sankaranarayanan. On the minimal revision problem of specification automata. *The International Journal of Robotics Research*, 34(12):1515–1535, 2015. doi:10.1177/0278364915587034.
10. Neeraj Kumar, Daniel Lokshtanov, Saket Saurabh, and Subhash Suri. A constant factor approximation for navigating through connected obstacles in the plane. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 822–839. SIAM, 2021. doi:10.1137/1.9781611976465.52.
11. Ao Li, Ankush Desai, and Rohan Padhye. Feedback-guided adaptive testing of distributed systems designs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 701–719. USENIX Association, 2026. URL: https://www.usenix.org/conference/nsdi26/presentation/li.
12. Nancy A. Lynch and Frits W. Vaandrager. Forward and backward simulations: I. untimed systems. *Information and Computation*, 121(2):214–233, 1995. doi:10.1006/inco.1995.1134.
13. Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi:10.1016/j.cosrev.2010.09.009.
14. Thomas Neele, Antti Valmari, and Tim A. C. Willemse. A detailed account of the inconsistent labelling problem of stutter-preserving partial-order reduction. *Logical Methods in Computer Science*, 17(3):8:1–8:27, 2021. doi:10.46298/LMCS-17(3:8)2021.
15. Noah Streib and William T. Trotter. Dimension and height for posets with planar cover graphs. *European Journal of Combinatorics*, 35:474–489, 2014. doi:10.1016/j.ejc.2013.06.017.
16. Levin N. Winter, Florena Buşe, Daan de Graaf, Klaus von Gleissenthall, and Burcu Kulahcioglu Ozkan. Randomized testing of byzantine fault tolerant algorithms. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):757–788, 2023. doi:10.1145/3586053.

A Complexity and proof boundaries

A.1 General finite transition systems

For the explicit finite LTS decision problem with binary-encoded budget C, guess a total fine policy and an allowed split set containing the required set. Check uniformity outside that set and $c(T) \leq C$, then perform ordinary reachability using only compatible edges. Summing binary-encoded nonnegative costs and these checks takes polynomial time.

For NP-hardness, reduce weighted Set Cover. Let $D_1, \ldots, D_m \subseteq U$ be a Set Cover instance with costs c_j, and allow every fibre to split with none required. Create two fine keys a_j, b_j in coarse fibre j. A forced initial chain records $(a_j, 0)$ for every set. Then place one layer per element $u \in U$; for every j with $u \in D_j$, add a parallel edge $(b_j, 1)$ to the next layer. The final layer is unsafe. Every path chooses covering sets and mixes exactly the chosen fibres, paying each selected set once. Conversely any cover supplies such a path of no greater cost. The graph is acyclic and uses exactly two fine keys per fibre. Thus the decision problem is NP-complete already under these restrictions, by the classical Set Cover hardness [8]. The artifact exhaustively checks the reduction on all 512 ordered three-set families on a three-element universe, independently enumerating covers.

The finite graph restriction is substantive. With no controlled keys, an unrestricted program can simulate a machine and reach an unsafe state exactly if it halts. A total solver deciding both reachability and infeasibility for all such programs would decide halting. The semantic simulation and lower-bound theorems themselves require no finite-state assumption; a supplied inductive invariant can cover an infinite concrete system.

A.2 Independent-pivot specialization

In the unweighted independent-pivot schema, the decision problem of whether a coarse unsafe execution exists is NP-complete when the number of pivots is unbounded and the finite opportunity order is represented explicitly by its events and precedence or cover edges. A certificate consists of an open set, a reusable set and a topological event order; checking admission, ordering and quorum support takes polynomial time.

For hardness, take an undirected graph on k vertices and an independent-set target t. Introduce repairs R_i and second-vote opportunities V_i. For every undirected edge $\{i, j\}$ impose $R_i \prec V_j$ and $R_j \prec V_i$. These constraints are acyclic: they point from the repair layer to the vote layer. The fixed prefix and certificate collectors can be added before and after those layers. The resulting repair graph is the symmetric orientation of the original graph. Its induced set is acyclic exactly when it is independent, since each undirected edge creates a directed two-cycle.

Set $d_0 = k, d_1 = t$. Proposition E3 gives coarse unsafe reachability exactly when $\alpha(H) \geq t$, which is the independent-set question. This uses the classical relation between node cover, feedback node sets, and symmetrized graphs [8]. It is a complexity statement about the declared opportunity schema. It neither establishes a complexity bound for the fixed four-validator trace nor claims that every constructed order is realizable by a production consensus implementation.

B Supporting replay and diagnostic results

B.1 Replay erasure with outputs

Let $\pi : S \to S_{\mathrm{rel}}$ project relevant state, including processed message keys. For every retained message m, require **state congruence** and **output congruence**:

$$\pi(s) = \pi(t) \Rightarrow \pi(\delta(s,m)) = \pi(\delta(t,m)), \qquad \pi(s) = \pi(t) \Rightarrow \mathsf{out}(s,m) = \mathsf{out}(t,m).$$

For an exact marked duplicate whose key was already processed, require projected stutter $\pi(\delta(s,m)) = \pi(s)$ and output silence $\mathsf{out}(s,m) = \epsilon$.

Supporting proposition (replay erasure). Deleting all marked duplicates preserves the terminal relevant projection and the complete relevant output trace under these conditions.

Proof. Maintain equality of the two current projections and accumulated outputs. A duplicate changes neither on the unerased side. At a retained message, the two congruences preserve the invariant. Induction gives the terminal claim. □

Projected silence alone is insufficient: in a trace X, X, Y, the duplicate can change a hidden bit that the later Y copies into relevant state or output. The artifact retains these negative controls. This proposition is standard congruence reasoning; it supports the distinction between internal state and persistent outputs without constituting the revised paper's main contribution.

B.2 The earlier seven-flag diagnostic

For Boolean flags $S_1, S_2, R_q, D, T, E_d, X$, the earlier operational abstraction uses

$$U(e,r) = (\neg S_1 \vee e) \wedge [\neg S_2 \vee ((\neg R_q \vee e) \wedge \neg(r \wedge D \wedge T))].$$

Here the shortage flags describe the two bases; R_q makes second participation enabling-dependent; D gives repair efficacy; T means repair precedes the **second-vote opportunity**; E_d couples the endpoint key; and X permits phase distinction. Direct substitution yields

$$X \wedge E_d \wedge U(1,0) \wedge \neg U(0,0) \wedge \neg U(1,1) \iff S_2 \wedge D \wedge T \wedge E_d \wedge X \wedge (S_1 \vee R_q).$$

Exactly three of the 128 assignments satisfy this expression. This is a diagnostic of that Boolean machine. The old atomic opportunity interpretation combined vote and certificate formation; it must not be reused to infer that repair merely before a later collector suffices. The revised frontier model separates these events and makes initially ready pivots the primary multi-pivot case.

C Detailed guarded four-validator model trace

This is the retained single-controlled-fibre witness. Here E is B's round-0 prevote delivered to P, and R is its round-0 precommit to P. The fine policy admits E and withholds R. The new coupled model additionally controls $A \to P$; its optimized execution is recorded separately in the artifact.

1. A receives B's block-0 proposal and signs $\mathrm{PV}(0,0)$. A relays the proposal to P, which signs the same prevote. C receives no proposal; its proposal timeout produces $\mathrm{PV}(0,\perp)$.

2. B emits its round-0 prevote and precommit. Deliver P's and B's prevotes to A. The set $\{A, P, B\} : 0$ makes A persist valid/locked block 0@0 and sign $\mathrm{PC}(0, 0)$.
3. Deliver A's prevote and E to P. Its same three-signer PoLC sets valid block 0@0. The mutation omits only its lock assignment, and P signs $\mathrm{PC}(0, 0)$ while remaining unlocked.
4. Deliver C's nil prevote to P, and the A, P block-0 prevotes to C. At C, the mixed set $\{A : 0, P : 0, C : \bot\}$ enables its prevote timeout without creating a block-0 PoLC. On expiry, C signs $\mathrm{PC}(0, \bot)$.
5. Deliver A's precommit to P, C, and P's precommit to A, C. Deliver B's precommit to A. Its local set $\{A, P, B\} : 0$ reaches three and A decides 0.
6. Deliver C's nil precommit to P. Both P and C now have the mixed set $\{A : 0, P : 0, C : \bot\}$. It enables a precommit timeout but does not commit block 0. The fine policy suppresses R. Expiry advances P, C lawfully to round 1.
7. P retains valid block 0@0 but no lock; C has neither. Consequently the unchanged proposer C can propose new block 1 with valid round -1. C and the unlocked P each sign $\mathrm{PV}(1, 1)$. Making P the proposer instead would require accounting for its cached valid block 0.
8. B emits round-1 prevote and precommit for 1. Deliver the C, B prevotes to P, producing PoLC $\{P, C, B\} : 1$. P normally updates valid/locked block to 1@1 and signs $\mathrm{PC}(1, 1)$. Deliver the P, B prevotes to C; it obtains the same PoLC and precommits 1.
9. Deliver the P, B precommits to C. Together with C's self-precommit, they produce decision 1. Delivering the C, B precommits to P also lets it decide 1. The unmutated observers have $A = 0, C = 1$.

D Parametric certificate families

For every $f \geq 1$, take $n = 3f + 1$, $q = 2f + 1$, Byzantine set B of size f, correct exclusive sets A, C of size f, and pivot p. Then

$$Q_0 = B \cup A \cup \{p\}, \qquad Q_1 = B \cup C \cup \{p\}$$

have size q and each nonpivot base has size $q - 1$. Under the behavioral contract, Proposition E2 applies. This arithmetic does not assert that every locking protocol realizes the contract.

More generally, for $1 \leq k \leq f$, use k pivots, exclusive sets A, C of size $f + 1 - k$, Byzantine set B of size f, and $k - 1$ unused correct identities. The total is still $3f + 1$. Both quorums contain B, their exclusive set, and all k pivots; each base has size $q - k$. Distinct correct observers exist in nonempty A, C. Proposition E3 characterizes precisely which opportunity orders can preserve these k required double contributions. The extension is a certificate schema; the guarded trace in Appendix C establishes the single-pivot model case.

E Structural special-case proofs

These results explain the independent-pivot specialization. The general transition-system optimizer does not assume their behavioral contracts. Here G_T denotes phase-specific admission and G_D coarse admission. Write $L_G(P, \mathcal{B})$ for legal executions under scheduler G and declared opportunity bound $\mathcal{B}$, and $\mathsf{UnsafeObs}_G(P, \mathcal{B})$ for observations of those executions whose terminal state satisfies the designated unsafe predicate.

E.1 A standard sufficient preservation condition

The useful positive baseline is a specialized simulation argument [12]. Let a candidate coarse trace retain the actions of a fine trace in order and insert closure-required deliveries. Suppose a preorder $\mathcal{R}$ relates the initial states, each insertion increases the target state in $\mathcal{R}$, retained actions preserve $\mathcal{R}$, and unsafety is upward closed in $\mathcal{R}$.

Proposition E1 (legal relational preservation). If the candidate trace $C(\tau)$ is legal in the coarse language, $C(\tau) \in L_{G_D}(P, \mathcal{B})$, these conditions imply $\mathsf{Unsafe}(\mathrm{last}(\tau)) \Rightarrow \mathsf{Unsafe}(\mathrm{last}(C(\tau)))$.

Proof. Induct over the merged action sequence. At an insertion, the target's increasing step composes with the current relation by transitivity. At a retained action, forward simulation advances both states. The terminal relation and upward closure transfer unsafety. Target legality supplies membership in the coarse execution language. □

Legality is an independent obligation. Processing repair may disable an action or prevent a message from being emitted, so simply retaining all fine votes after inserting repair can produce an illegal sequence. Our negative results instead rerun guarded opportunities; a disabled opportunity emits no vote. Proposition E1 is supporting machinery, not a new general simulation principle.

E.2 A corrected one-pivot separation criterion

Let $q \geq 1$ be a certificate threshold. Two conflicting certificate families have fixed nonpivot support $q - 1$ each. The same pivot p is the only possible additional signer. The first and second votes are issued at opportunities V_0, V_1; C_0, C_1 collect their certificates. Decisions, signed votes, and the availability of emitted votes at the specified collectors are irreversible within the bound.

The following are **behavioral obligations**:

1. **Enabling necessity.** The first pivotal vote is emitted exactly when the enabling envelope E is admitted. No substitute supplies that missing contribution.
2. **Explicit persistence.** When E is admitted and repair R is withheld, p emits the second pivotal vote at V_1. Other fixed events do not independently disable it.
3. **Timely repair.** When E, R are admitted and R is processed before V_1, p does not emit the second pivotal vote. No previously emitted second-family signature can substitute for it.
4. **Certificate and observation coverage.** The two designated correct observers decide their respective values exactly when the corresponding quorum is available; the bounded agreement observation is unsafe exactly when both families decide. No unrelated decision opportunity creates an additional disagreement.

The **scheduler obligations** are separate: E, R are emitted, share one endpoint/view key, and have distinct phase labels. Fine admission can realize $(e, r) = (1, 0)$; coarse admission realizes only $(0, 0)$ and $(1, 1)$. The relevant order is

$$E < V_0 < C_0 < R < V_1 < C_1.$$

All other delivery choices and opportunities are fixed, and each rerun obeys the behavioral guards. These assumptions deliberately describe a sufficient schema; quorum arithmetic alone cannot establish a protocol refinement.

Proposition E2 (behavioral quorum-pivot separation). Under these behavioral and scheduler obligations,

$$\mathsf{UnsafeObs}_{G_T}(P, \mathcal{B}) \neq \varnothing, \qquad \mathsf{UnsafeObs}_{G_D}(P, \mathcal{B}) = \varnothing.$$

Proof. Fine admission supplies the first vote by enabling necessity and the second by explicit persistence. Each augments a base of $q-1$, producing two conflicting correct decisions. With the coarse edge off, the first vote is absent and its certificate is short, regardless of whether p can still vote second. With the edge on, the first certificate forms but timely repair prevents the second pivotal signature. Observation coverage makes these two coarse cases exhaustive for agreement. □

The persistence obligation cannot be inferred from the absence of R. For example, a fixed local transition after C_0 could disable p even when R is suppressed. The endpoint and quorum conditions would still hold, but the fine witness would disappear. Likewise, the order $R < C_1$ is insufficient if $V_1 < R < C_1$: repair cannot retract the already signed vote. Both errors disappear when behavior and signature timing are stated explicitly.

The familiar $n = 3f + 1$, $q = 2f + 1$ family instantiates the two bases with $q - 1$ fixed signatures. Appendix D gives its arithmetic and a k-pivot extension; realizability of their behavior remains a separate obligation.

E.3 Repair frontiers for several independent channels

Let $I = \{1, \ldots, k\}$ be pivotal identities, each with positive integer weight w_i. Write $w(X) = \sum_{i \in X} w_i$ and $W = w(I)$. The two certificate families have fixed base weights b_a, thresholds q_a, and deficits $d_a = \max(0, q_a - b_a)$, for $a \in \{0, 1\}$. Unit weights give $W = k$.

E.3.1 Behavioral and order contracts

Pivot i emits exactly one first-family signature if and only if E_i is admitted. No substitute first-family contribution is available when it is closed. Once the first family is feasible, pivot i emits its second-family signature at opportunity V_i **if and only if** R_i has not been processed before V_i. In particular, closing its edge need not prevent its second vote: it was initially ready. No other state transition changes this test. On each first-certificate-feasible execution, an admitted repair is already available and is processed at its designated R_i slot; admission is not an unproduced-envelope stutter. An admitted E_i likewise realizes its enabling action. These availability and slot-coverage conditions are part of this specialization. Admitted repair disables prospective issuance; it cannot withdraw signed output. Both collectors and the observation satisfy the coverage obligation of the behavioral separation subsection.

It suffices to impose this second-vote law on policies capable of forming the first certificate. A policy that misses that certificate cannot be unsafe in this two-family observation domain. This qualification accommodates repairs whose commit evidence depends on available first votes.

A finite strict partial order $\prec$ describes opportunities. Repair and vote slots are distinct event kinds: $R_i \neq V_j$ for all i, j. The fixed anchors include

$$E_i \prec V_0 \prec C_0 \prec V_i \prec C_1, \qquad V_0 \prec R_i.$$

Additional relations specify the allowed delivery restrictions. Each policy admits **all** linear extensions of this order, interpreted through the guarded machine. Withheld deliveries and

disabled voting opportunities stutter. Their slots retain the stated precedence constraints; suppression does not erase a transitive constraint between other events. Every processed envelope must already be emitted and available.

The k endpoint/view keys are independent. A coarse policy opens a subset $A \subseteq I$ and admits E_i, R_i together exactly for $i \in A$. Fine admission can admit all E_i and suppress all R_i. These are explicit independence and order-coverage assumptions. If a real harness admits fewer orders, the model gives a sound coarse-safety overapproximation, but an existential witness needs a realizable order in that harness.

E.3.2 A graph of simultaneous repair obstructions

Define a directed graph H, including possible self-loops, by

$$i \longrightarrow j \quad \Longleftrightarrow \quad R_i \prec V_j, \qquad \alpha(H) = \max\{|J| : J \subseteq I,\ H[J] \text{ is acyclic}\}.$$

Use transitive precedence, not just a drawing's cover edges. A self-loop means that a pivot's own repair is forced early.

Lemma E1 (compatible late repairs). For $J \subseteq I$, a linear extension can place $V_i < R_i$ for every $i \in J$ exactly when $H[J]$ is acyclic.

This is the standard reversible-pair/alternating-cycle argument from partial-order theory, expressed in repair notation [15]. We include the proof because the mapping to scheduling opportunities is central.

Proof. Add the required edges $V_i \to R_i$ to the opportunity order. Such an order exists exactly when the augmented relation is acyclic: a finite acyclic relation has a topological ordering. A cycle $i_1 \to \cdots \to i_t \to i_1$ in $H[J]$ yields an augmented closed walk

$$V_{i_1} \to R_{i_1} \prec V_{i_2} \to R_{i_2} \prec \cdots \prec V_{i_1}.$$

Conversely, any augmented cycle must use an added edge, since the original order is acyclic. Between consecutive added edges $V_i \to R_i$ and $V_j \to R_j$, the original-order segment gives $R_i \prec V_j$, hence $i \to j$. Compress these segments to obtain a nonempty closed walk in $H[J]$, which contains a cycle. This includes self-loops. □

The graph can have cycles even though the opportunity order does not. Its cycles describe incompatible *requests to delay repair*, not a causal cycle in an actual execution.

The full lemma is machine-checked, including existence of the finite topological order. The Lean construction assigns an injective time using the number of strict predecessors and an enumeration index. A separate theorem accepts cover edges and derives the graph from their transitive closure. Thus neither realizability nor the treatment of suppressed slots is an assumed capacity premise.

E.3.3 Exact capacity and separation

Proposition E3 (repair-frontier characterization). Under the contracts above, fine unsafety is feasible exactly when $d_0, d_1 \le W$. Coarse unsafety is feasible exactly when there exist $J \subseteq A \subseteq I$ satisfying

$$H[J] \text{ acyclic}, \qquad w(A) \ge d_0, \qquad w(I \setminus A) + w(J) \ge d_1. \tag{E1}$$

For unit weights this is equivalent to

$$\mathsf{UnsafeObs}_{G_D} \neq \varnothing \quad \Longleftrightarrow \quad \max(d_0, d_1) \leq k \ \wedge \ d_0 + d_1 \leq k + \alpha(H). \tag{E2}$$

Consequently strict phase/coarse separation holds exactly when

$$\boxed{\max(d_0, d_1) \leq k \quad \wedge \quad d_0 + d_1 > k + \alpha(H).} \tag{E3}$$

Proof. Fine admission can supply all first and second signatures by admitting every enabling envelope and suppressing every repair. No branch can receive more than weight W, proving its characterization.

For a coarse unsafe execution, let A be its open keys and let J be those open pivots whose second votes precede their repairs. The first support is exactly $w(A)$. Because the first certificate is feasible, the second-vote contract applies: closed pivots remain ready and open pivots contribute second exactly when in J. Thus the second support is $w(I \setminus A) + w(J)$. The execution realizes their joint lateness, so Lemma E1 gives acyclicity and establishes necessity of (E1).

Conversely, choose A, J satisfying (E1). Lemma E1 gives an order in which every member of J signs before repair. Open precisely A. The first threshold is met, so the behavioral contract ensures second signatures from every closed pivot and every member of J. Further signatures only increase support. Durable outputs reach both collectors; observation coverage gives an unsafe execution.

For unit weights, adding $d_0 \leq |A|$ and $d_1 \leq k - |A| + |J|$ proves the combined inequality in (E2); each demand is at most k. For the converse put $h = \max(0, d_0 + d_1 - k)$. Equation (E2) implies $h \leq \alpha(H)$ and $h \leq d_0 \leq k$. Choose an acyclic set of size h, possible by taking a subset of a maximum acyclic set, and extend it to an open set A of size d_0. Then $k - |A| + h \geq d_1$, so (E1) applies. Combining fine feasibility with the negation of (E2) gives (E3). □

Here h is the number of pivots that must contribute to **both** branches. Coarse reachability is decided by whether the scheduler can keep that many jointly reusable contributors, not by the number of admission bits alone.

E.4 Robustness and the amount of phase control required

E.4.1 Common deadlines and crossed deadlines

If all second signatures share one opportunity V, let $F = \{i : R_i \prec V\}$ and $\ell = k - |F|$. Members of F have self-loops; all other pivots can jointly place repair after V. Hence $\alpha(H) = \ell$ and (E2) becomes $d_0 + d_1 \leq k + \ell$, subject to individual feasibility. For the k-required family, $d_0 = d_1 = k$: separation survives whenever $\ell < k$. Thus allowing some late repairs need not eliminate separation once more than one double contribution is indispensable.

Individual deadlines behave differently. With two pivots and only the critical constraints

$$R_1 \prec V_2, \qquad R_2 \prec V_1,$$

each repair can individually follow its own vote. They cannot both do so: that would require $V_1 < R_1 < V_2 < R_2 < V_1$. The repair graph is a directed two-cycle and $\alpha(H) = 1$, despite having no self-loop. With $d_0 = d_1 = 2$, coarse admission remains safe and phase-sensitive admission is unsafe. This is a separation over several event orders and two independently controlled channels, rather than two executions of one fixed edge.

E.4.2 Optimal phase-splitting budget

A split key admits its enabling phase while suppressing repair. Splitting leaves the opportunity partial order intact. With a chosen split set T, the maximum number of double contributors is

$$|T| + \alpha(H[I \setminus T]).$$

Among unsplit keys, Lemma E1 still applies. A split pivot contributes twice without requiring its repair to be late. Choosing at most t split keys gives the exact best capacity $\min(k, \alpha(H) + t)$: this is an upper bound because any unsplit acyclic set has size at most $\alpha(H)$; it is attained by keeping a maximum acyclic set and splitting vertices outside it.

Corollary E1 (minimum phase budget). For unit weights with no required splits and every key available at unit cost, if fine admission is feasible, the exact minimum number of independently split keys needed for an unsafe witness is

$$\boxed{t_* = \max(0, d_0 + d_1 - k - \alpha(H)).} \tag{E4}$$

Proof. Substitute the best capacity $\min(k, \alpha(H) + t)$ into (E2). Since $d_0, d_1 \le k$, the resulting inequality holds exactly when t covers the deficit in (E4). □

Figure 1 in Appendix F illustrates the unit-weight boundary. The statement is existential over the choice of split keys. It does not claim that every set of t_* arbitrary message exceptions suffices. A witness can be constructed by choosing an acyclic reusable set, selecting the necessary split keys and open keys, adding its lateness constraints, and topologically sorting the opportunity graph.

E.4.3 Order relaxation, extra edges, and weights

Removing precedence constraints cannot add an edge to H, so it cannot decrease $\alpha(H)$. Thus order relaxation weakens separation monotonically under the same behavioral contract and deficits. Conversely, additional causal paths may increase the needed phase budget. Independent auxiliary events and extra edges are harmless only if they preserve both relevant state/output behavior and the stated order language. An extra substitute vote changes a deficit and must be included in the calculation.

Initially ready and enabling-dependent pivots are different models. With two forced-early pivots and deficits $1, 1$, coarse admission can open one key for the first vote and close the other for the second. Requiring both pivots to see enabling evidence before their second votes would incorrectly remove that coarse witness. Proposition E3 uses the initially ready behavior present in the four-validator edge-off case.

For a common deadline with integer weights, define $L = I \setminus F$ and $W_L = w(L)$. The exact condition is existence of $K \subseteq F$ such that

$$\max(0, d_0 - W_L) \le w(K), \qquad w(K) + d_1 \le W. \tag{E5}$$

Open every key in L and delay those repairs; choose K among the forced-early keys. The two supports become $W_L + w(K)$ and $W - w(K)$. Opening all L cannot hurt either branch, which proves necessity as well as sufficiency. Indivisible weights matter: weights $2, 2$, no late repair, and deficits $1, 3$ require an unattainable subset weight 1. A total-weight relaxation

would give the wrong answer. This is an abstract weighted certificate result, not a weighted CometBFT refinement.

For arbitrary individual deadlines, deciding whether a coarse unsafe execution exists is NP-complete for explicitly represented finite opportunity orders with unbounded k. Appendix A gives the short reduction using a classical graph construction [8]; the contribution is the scheduler correspondence, not a new graph-hardness result.

E.5 Construction and independent certificates

Compute H using reachability in the full opportunity graph. Vertices with self-loops cannot belong to J; treat them as individual blocks, then partition the remaining graph into strongly connected components. Every directed cycle of a candidate $H[J]$ lies within one remaining component. Enumerate local assignments to the four roles in the weighted four-role characterization, respecting allowed and required keys, and reject cyclic reusable sets. Record each assignment's guaranteed supports (u, v) and cost c; supports may be capped at the respective deficits.

Let blocks be $C_0, \dots, C_{m-1}$ and their local alternatives be $\mathcal{A}_j$. Dynamic programming over residual deficits gives

$$L_j(a,b) = \min_{(u,v,c)\in\mathcal{A}_j} \left[c + L_{j+1}\big((a-u)^+, (b-v)^+\big)\right], \tag{E6}$$

with $L_m(0,0) = 0$ and $L_m(a,b) = \infty$ otherwise. Backtracking selects roles, admissions and the requested reversals; topological sorting produces a complete event order. A separate event interpreter checks the actual immutable-output execution and its quorum support.

The portable certificate contains the input, chosen execution and potential table. Its checker independently reconstructs transitive reachability, components and **all** legal local assignments. It checks the lower-bound inequalities

$$L_j(a,b) \le c + L_{j+1}\big((a-u)^+, (b-v)^+\big) \tag{E7}$$

for every local transition, together with the terminal conditions. Checking only the optimizer's chosen assignments would not certify minimality. Feasibility is checked by replay; the measured witness cost must equal the certified root bound. An external expected input can be supplied so that a valid certificate for another problem is rejected.

Proposition E4 (certificate soundness). If all conditions in (E7) and the terminal conditions hold, every feasible refinement represented by Theorem 6 costs at least $L_0(d_0, d_1)$. A replayed witness attaining a finite root bound is minimum-cost. An infinite root bound certifies infeasibility.

Proof. Follow the local assignments of any feasible refinement through the residual-deficit states. Repeatedly applying (E7) bounds the initial potential by the accumulated cost plus the terminal potential, which is zero. An infinite potential cannot reach a finite terminal along a finite-cost path. Matching the lower bound with a feasible witness proves optimality. The completeness of the four-role representation, established in Theorem 6, connects this path argument to the scheduling problem. □

Lean checks this Bellman principle for arbitrary nonnegative-cost transition systems, using extended natural bounds. It does not assume that the potentials were computed optimally. The Python parser, graph extraction and exhaustive local enumeration remain executable proof obligations. This follows the established certifying-algorithm approach [13].

Enumeration is exponential in the largest remaining component, and the potential grid has $(m+1)(d_0+1)(d_1+1)$ cells. The dependence on integer quota values is pseudo-polynomial; verification still enumerates local configurations. Neither compact polynomial certificates for all instances nor a polynomial-time optimizer are claimed. Explicit component, table and time limits return a resource-limit result, never a false infeasibility claim.

F Unit-weight budget illustration

The heat maps instantiate Corollary E1 at three reusable capacities. They illustrate the cardinality special case; unequal control costs require the optimization in Theorem 6.

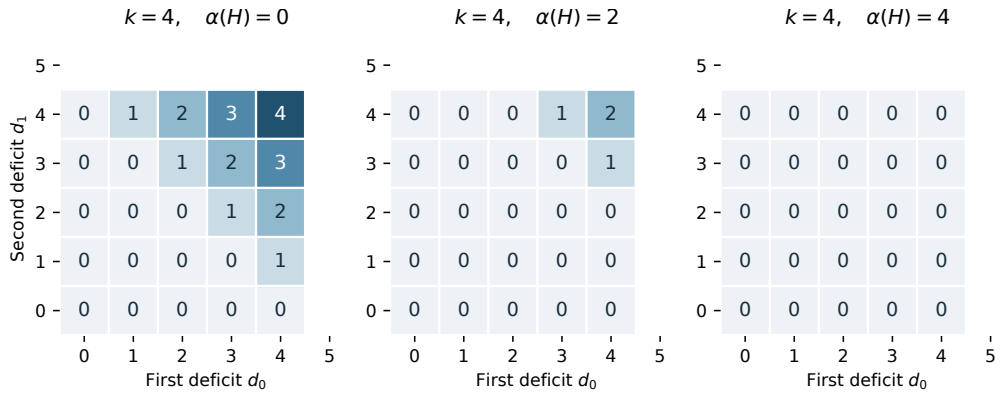


Figure 1 Exact phase-control budget in the independent-pivot schema. Each cell is the minimum number of phase-split keys required to expose an unsafe witness. Blank cells exceed the available pivot count. Panels differ only in the capacity of jointly late repair, not in quorum size.

G Seven-validator instance and comparison protocol

G.1 The declared protocol and environment

All validators have voting power one. The rule-following set is $\{A, P_0, P_1, C, D\}$; B_0, B_1 execute a fixed Byzantine message script. Height 1 has rounds 0 and 1 and valid blocks 0 and 1. B_0 proposes in round 0 and C in round 1. The mutant omits only the round-0 lock assignment at P_0, P_1. These are defective implementations of the guarded rule; the experiment is not a disagreement under the original protocol's fault assumptions. The baseline and single-mutation variant preserve the corresponding lock assignments. Every decision requires a matching local five-signer certificate and the matching block. Entering commit stops further signing even while block data is missing.

The six coarse fibres are the round-0 vote routes $A, B_0, B_1 \to P_i$, for $i \in \{0, 1\}$. Each contains exactly PREVOTE and PRECOMMIT admission keys. A total policy has twelve persistent Boolean answers; a split permits its two answers to differ. The main costs are all 1; the weighted profile assigns $(1, 2, 3)$ to the three routes at each pivot. Other messages follow the fixed environment. In particular D receives and relays the first proposal to the pivots, so no uncontrolled proposal traverses a controlled fibre. Undeclared message types on a controlled endpoint are rejected by the adapter.

The finite environment has 16 ordered stages. Stages 0–1 bootstrap proposals, Byzantine messages and fixed prevotes; stages 2–4 offer the two pivot deliveries of each sender's controlled prevote, in either order. Stage 5 supplies mixed-prevote support and guarded timeouts; stage

6 delivers fixed precommits. Stages 7–8 offer the controlled A and B_0 precommits. Stage 9 offers $B_1 \to P_0$'s precommit and P_0's timeout in either order; stage 10 does the same for P_1. Stage 11 offers C's timeout. Stages 12–15 execute guarded round-1 proposal, prevote, timeout and precommit batches. The 23 slots are once-only. A missing message or unarmed timer yields a conditional no-effect opportunity, not an invented protocol event. Every permutation within each variable stage is represented, while fixed batches execute their primitive events in the declared order. This is a bounded input language, not a partial-order-reduction theorem covering all interleavings.

The retained state includes local rounds, steps, locks, valid blocks, proposal metadata, block availability, commit and decision fields, signer masks and armed timers, together with emitted votes, consumed envelope slots and proposal possession. Audit timestamps and log history are excluded. Deterministic HMAC test signatures are reconstructed from the retained signing slots during replay. The model does not implement native Ed25519, block-part retry, dynamic membership, arbitrary Byzantine equivocation, future-round jumps, networking threads or unbounded heights. The pinned source correspondence is the same as in Section 6.1; production-code coverage remains a separate obligation.

G.2 Causal explanation and controls

The first certificate at A needs the set $\{A, P_0, P_1, B_0, B_1\}$ voting for 0. Each pivot's matching prevote quorum therefore needs all three controlled incoming prevotes together with self and cross-pivot support. Each pivot later receives its own and the other pivot's precommit for 0 and the two nil precommits from C, D. At least one controlled precommit is needed to reach the any-value quorum of five and arm the timeout. Admitting all three instead completes five matching precommits for 0, forcing commit entry. Hence an unsafe continuation needs one or two, but not zero or three, of those precommits at each pivot, with the enabling delivery before the once-only timeout when necessary.

After lawful round advancement, the unlocked pivots can support C's proposal 1 despite retaining valid block 0. C has no old valid block to reuse. The second matching certificate needs $\{C, P_0, P_1, B_0, B_1\}$. Both pivot signatures are indispensable. All six enabling prevotes must be admitted, and each pivot must suppress at least one precommit on a route whose prevote was admitted. Thus at least one split per pivot is necessary, giving unit-cost lower bound 2. Splitting both A routes and retaining the four Byzantine routes attains it. This explanation is checked independently by optimizing and certifying the emitted transition graph; the exporter is not given the target cost.

Complete bounded model	States	Edges	Unsafe
Both pivot lock assignments omitted	48,201	55,283	36
All locks retained	48,201	55,283	0
Only P_0 lock assignment omitted	48,201	55,283	0
Cross-pivot round-0 prevote removed	50,411	57,374	0
C, D nil-precommit support removed	44,363	51,630	0
$P_1 \to C$ round-1 prevote removed	48,201	55,283	0

Table 3 Guard and dependency controls. Removal changes a declared fixed delivery opportunity; it does not overwrite a guard or the unsafe predicate. Zero unsafe states concerns these complete bounded graphs.

For a timing control, admit all controlled prevotes and only each B_1 precommit. Delivery-first ordering in stages 9–10 reaches $A = 0, C = 1$; timer-first ordering spends unarmed

opportunities and leaves only $A = 0$. Both executions use the identical static policy. Separately, with all admissions true, timer-first ordering is enabled by earlier A, B_0 precommits. The late matching B_1 precommit then enters old-round commit waiting for missing block 0; signing cannot resume. The model does not supply a later block-fetch opportunity. These replays distinguish lack of repair, lack of progress, certificate availability, commit entry and final decision.

G.3 Search, proof and measurement boundaries

The control-flow-only partition retains stage, remaining slots and unsafe observations, yielding 32 states and 62 edges. Its minimum 0 has no concrete witness under the returned policy. The guard-aware seed additionally retains each pivot's count of locally received round-0 prevotes for 0 and its emitted round-0 precommit. It has 449 states. The map never inspects costs, allowed splits, candidate policies, the optimum or a witness. It was developed after a slow flow-only pilot and is reported as an exploratory feature choice. Both starts retain the same complete concrete graph; all-edge simulation is checked independently.

Every intermediate abstract certificate, rejected policy invariant and strict partition split is retained and checked. The final all-edge map and a concrete witness of matching cost suffice for the final optimum independently of that search history. Counts distinguish the sum of all product searches, peak states within one solver invocation, final strict-sublevel proof states, concrete policy exploration, all-edge coverage and safe-policy closure scans. Deterministic compressed sizes distinguish the shared concrete input, the final bridge and the full history. None of those individual quantities is total resident memory.

An ordinary graph-reachability oracle enumerates total policies independently of product memory and cost search. The corpus is reused across cost profiles and all allowed-fibre subsets; those reused queries are not additional protocols. Fresh guarded explorations of selected policies and primitive witness audits separately challenge the exporter. They share the underlying guard implementation and do not constitute a second independently implemented consensus specification.

The main unit-cost comparison uses three repetitions with alternating method order. Other cost, permission and guard controls are checked once. Timings charge concrete graph generation to both methods, partition construction to refinement, complete answer construction, external verification and primitive replay. Oracle enumeration, serialization and disk writes are excluded from both totals. Source hashes, all samples and platform information accompany the observations. The flow-only sensitivity has a fixed eight-refinement limit; a separate comparison imposes the same 10,000-product-state cap per solver invocation. Multiple refinement invocations can use more cumulative work and memory. A cutoff remains unknown regardless of what the independent oracle says. Graphs with an empty unsafe set are guard controls, not evidence of general speed superiority: a specialized direct checker could exploit that emptiness immediately.